

NOW — one ordered list

Everything remaining, merged into a single priority order. Blockers, code, deploy, pitch. Work top to bottom. Notes explain why each item sits where it does, so the order can be argued with rather than followed blindly.

Status at the time of writing: 84 tests passing. All 11 checks, replay, dual control, oversight hold, revoke, the Monad anchor and negotiation are all built. What is left is mostly **not code**.



The submission is a gate, not a task. It is not item 20 on this list. Submissions close 12:00 Sunday and **pitching order follows submission order**, so submit the moment the deploy is verified, then keep polishing. Everything below is ranked for what to do with the hours you have, not for what to leave until last.

0. Sunday morning, before anything else: make the repo public

Deliberately private overnight so the idea is not lifted before judging. That is a fine call, but it is also a **submission blocker with a 30-second fix**, and Sunday morning before a noon deadline is exactly when a 30-second task gets forgotten.

GitHub → the repo → **Settings** → **Danger Zone** → **Change visibility** → **Public**.

Then check it from a logged-out browser, because the failure mode is silent: it looks fine to you and 404s for everyone else. The deployed footer and the submission both link there.

- Repo public
- `github.com/theomthakur/raingentic-team10` loads in a private window

1. Walk over and ask a Rain engineer three questions

Ranked first because it is the only item that **depends on other people and expires**. The engineers are on site today. Everything else on this list is still doable at 2am; this is not. Three one-line answers unblock finished code.

1. **Collateral.** `GET /issuing/users/{userId}/contracts` returns `[]`, and `GET /contracts/{id}` on the ID from our sheet returns 403. Does it need attaching, or funding with RUSD? → *This is the only thing between us and money genuinely moving.*
2. **Card status enum.** `PATCH /issuing/cards/{id}` exists but `{"status":"inactive"}` returns 400. What value deactivates a card? → goes straight into `RAIN_CARD_INACTIVE_STATUS`, revoke then works with **no code change**.
3. **The configuration schema** for a spend limit and short expiry. It comes back as `{"currency":"usd"}`, and Rain's *minimum* body produces an active card with **no limit and a six-year expiry**.

 **Note:** ask #1 first. If collateral cannot be linked today, cards stay simulated, and that is fine — the demo's headline moment is a purchase that was *stopped*, so issuance is the only thing that has to work. Knowing at 3pm is worth far more than discovering at 8pm.

 **Note:** while you are up, do item 4 in the same trip.

2. Get a database and set `DATABASE_URL`

Three minutes, and it protects the one feature that cannot be cut.

Free Postgres at neon.tech → copy the connection string → into `.env.local` now, and into Vercel at deploy time (item 6).

 **Note, this is the trap:** Vercel is serverless, module memory does not survive a cold start. With no database the append-only log **empties whenever the instance goes idle**, so replay has nothing to replay. It works perfectly on a laptop either way, which is exactly why it gets missed. The app shows a red banner in production without it. Do not ship with that banner showing.

Currently empty in `.env.local`. Tables self-create on first request and the 60 seeded decisions load automatically, so there is nothing to migrate.

3. Delete the `crossval-pricing` comment in `lib/money.ts`

Thirty seconds. Line 1 references another project by name, on a **public repo**, judged under "all code written fresh today." Reword or delete.

 **Note:** ranked this high purely because the cost is zero and the downside is a rules question you do not want to be answering. Not a code risk, a *credibility* risk.

4. Ask the other teams what they are building

Perishable intel, and nearly free. Do it on the same walk as item 1.

 **Note:** expect two or three teams in the "spending guardrails / policy layer" category — it is the modal answer to this brief and `WHAT-WINS.md` predicted it. Knowing *who* at 3pm means the pitch can carry one sentence only we can say. Knowing at the demo is useless.

5. Merge `iterations` → `main`

Blocks everything visual, and gets **strictly worse** the longer both of you keep editing `app/page.tsx`.

```
git checkout main && git pull      # local main is 3 behind the remote
git merge iterations
npm test && npm run build
```

 **Note, the handoff doc is out of date on this.** `HANDOFF.md` says "8 commits ahead of main." It is now a **two-way split**: `origin/main` has 4 commits `iterations` does not have (agent personalization, vendor avatars, negotiation transcript, PDF receipt, agents roster page, flow diagram, footer). This is a bigger merge than described.

 **Resolution rule:** keep A's light-theme styling, keep `iterations` ' functionality. If a component on `main` looks visually newer but is missing props like `onAnchor`, `ruleChanges` or `onPropose`, take the `iterations` logic and re-apply `main` 's `classNames`.

Expect conflicts in `app/page.tsx`, `components/*`, `tailwind.config.ts`.

6. Deploy, and verify it the pedantic way

There is **no deployed URL anywhere in the repo yet**. Locally hosted submissions are disqualified outright, so until this exists there is no submission.

- ☐ `git ls-files | grep -i env` returns **only** `.env.local.example` — run this *before* the repo is public, not after. A public repo with a leaked key is a live incident.
- ☐ Repo is actually public, not "judges added"
- ☐ `DATABASE_URL` set in Vercel env vars
- ☐ **No red banner** on the deployed page
- ☐ Deployed URL loads on a **phone, on cellular**, not the laptop that built it
- ☐ Run a task, run it again → second one refused. **On the deployed URL.**
- ☐ Change a rule, hit replay → real numbers, meaning the seeded history loaded
- ☐  **Wait ten minutes, reload. Are the decisions still there?**
- ☐ Press "Reset demo" so the next person starts clean

 **Note:** that ten-minute item is the one to be pedantic about. Every other failure on this list is loud. This one fails by looking *empty*, and it fails on the exact link a judge opens.

7. Rehearse the demo, out loud, timed, twice

Higher value per minute than any remaining code. The order is in `SUBMISSION.md §4`.

The two moments that land: **step 2** (press the same button again — refused by the record the first run itself wrote) and **step 6** (propose a rule change, then try to activate it under your own name — refused).

 **Telegraph step 2 before you do it.** "Watch — I'm going to press this exact same button a second time." Announced, it reads as a designed property. Unannounced, it reads as a bug.

 **Close by naming the three refusals:** "You've now watched our own system refuse us three times — a duplicate purchase, a \$43,500 escalation, and my own rule change. None of them were scripted." Nobody who built this in an afternoon has a demo where the presenter gets blocked three times. It is free and it is the signature.

8. Drill the six hostile answers, one breath each

This is the real test of whether the idea is deep enough — not "is there a better idea." Anything that needs a paragraph is the weak spot.

Question	Answer
Isn't this just a foreign key constraint before the API call?	It's a three-way match , the control your AP team already runs, moved from after the invoice to before the instrument exists.
Why not just use Rain's own controls?	They bound <i>how much</i> and <i>where</i> . We bind the card to the obligation .
Couldn't the agent just lie?	It can. That's the assumption. It can't make the record agree with the lie.
What if two requests arrive at once?	They did, and both were approved. Here's the fix and the test.
What stops you editing the rules to fit the history?	Two people, and the version hash is on Monad before the decisions that cite it.
How would this scale?	<i>(point at the out-of-scope lane on the diagram)</i>

 **Note, use the bug deliberately.** Iteration 7: two identical requests fired at once, both approved, two cards issued. Found it, closed it with an atomic order-line claim, wrote three tests. **Volunteering a bug you found in your own system is the most credible thing available in a hackathon**, because nobody fabricates a bug. One of the judges builds high-throughput transactional systems for a living — a double-click, a retried webhook, two workers draining one job is his Tuesday. Say it as: *"we claimed idempotency, then tested it under concurrency and the claim was false. Here's what it took to make it true."*

 **Note, volunteer the limitation before it's found.** Cards are simulated because no collateral is linked. Say so, show the call, name the open question. Admitting the gap is what makes everything else you claim believable — and one judge has won hackathons himself and will know what was faked.

9. Show the rule `basis` on screen

Confirmed **not rendered anywhere**. Every rule already carries a `basis` string naming the real control it implements — three-way match, idempotency key, delegation of authority. It is in the data and in the docs and invisible to a judge.

`components/RuleEditor.tsx`, small muted line under each rule label.

 **Note:** ten minutes, and it is most of the trust argument. The honest answer to "*why would I trust software to spend my money*" is not "our rules are clever," it is "these are the controls your finance team already runs, moved earlier." That only works if it's on the screen. **The one exception to "do not build anything else."**

10. Monad: RPC URL + funded testnet key

Both empty. `lib/monad/anchor.ts` is finished and wired — the anchor button appears the moment the credentials exist. No code to write.

 **Note:** anchor the **rule version**, not each decision. That is the structural one: it proves the rules were not rewritten afterwards to fit a history you already had, which replay alone does not prove. Only *active* versions can be anchored.

11. Rain card enum → real revocation, and the stray card

Once item 1 answer #2 is in hand: put it in `RAIN_CARD_INACTIVE_STATUS` and revoke goes from simulated to real, no code change.

 **Note on the stray card** `ab3ea8c1-b0f3-4409-a7fb-a351e6a4d3ce` — active, unscoped, expires 2032, created accidentally while mapping the API. Two options, both fine: deactivate it, **or keep it deliberately as the demo's foil**. An unscoped six-year card sitting next to a Mandate card scoped to one PO and expiring with its quote makes the argument better than any slide. If you keep it, say clearly that you created it by accident — that's the story.

12. Optional, in this cut order

12a. **"Run all agents" button** — confirmed not built. `RunPanel.tsx`, sequential `await` per task.  **Never concurrent on the same PO** — that path is now deliberately refused (iteration 7), so a parallel version would demo your own guard rail failing.

12b. **Per-decision Monad anchor** — cut this before anything else. The rule-version anchor is the one that carries the claim.

The two calls worth repeating

Merge before writing any new feature. Two people editing `app/page.tsx` for another hour makes item 5 strictly worse, and item 5 gates the deploy.

Do not add a feature to differentiate. That instinct is exactly what lost ABI with 85% of the winning system's substance. Items 7 and 8 beat items 9 through 12 combined, and neither needs a line of code.